Meituan

Meituan-Robotics-0: A Vision-Language-Action Foundation Model for Desktop Manipulation

Meituan Robotics Team

Abstract

General-purpose desktop manipulation requires an embodied system that can acquire broad behavioral priors from heterogeneous data, transfer them across robot embodiments, recover from closed-loop execution errors, and preserve task-relevant state over extended horizons. We present Meituan-Robotics-0 (MR0), a vision-language-action foundation model and embodied-agent system for stationary-base desktop manipulation. Its data foundation combines 147 million vision-language samples with 15,060 hours of curated robot trajectories spanning 12 embodiments. MR0 converts this heterogeneous experience into executable control through a three-stage training recipe. Discrete action-token pre-training first aligns multimodal representations with robot behavior. Continuous flow-matching mid-training then transfers these priors to fine-grained cross-embodiment control. Real-world post-training further adapts the policy using successful demonstrations, human-gated corrections, and outcome-labeled autonomous rollouts. We introduce Rollback, a data collection mechanism that pairs replaced policy segments with human corrections for CFG-DAgger training. A unified end-effector interface supports learning across heterogeneous embodiments, while asynchronous action-chunk merging maintains smooth execution under variable inference latency. Because the low-level policy is reactive, an event-driven agent converts visual observations into persistent task events and invokes task-level reassessment only when task progress or feasibility changes. This separation preserves continuous motor control while enabling cross-task retrieval and conditional task resumption. Evaluated as a standalone policy, MR0 achieves 93.6% success on RoboTwin 2.0 and ranks fifth in the RoboDojo VLA track. Under human-gated DAgger post-training, MR0 improves on letter-block placement and garment folding compared with $\pi_{0.5}$. On Ethernet insertion, CFG-DAgger trained with Rollback data reaches 70% success and a progress score of 0.90 without value-model training or advantage estimation. Qualitative agent deployments further demonstrate cross-task event retrieval and recovery from temporarily unsatisfied task conditions. These results show that aligned heterogeneous data, real-world post-training, and event-level memory provide complementary foundations for transferable and long-horizon desktop manipulation. Code, datasets, and model checkpoints will be released at here.

1 Introduction

Vision–language–action [9, 50, 36, 42, 47, 35, 12] models provide a general interface for mapping visual observations and language instructions to robot actions. Their increasing data and model scales have substantially broadened the range of manipulation behaviors that can be learned within a single policy. Deploying such policies on desktop manipulation, however, exposes several limitations that cannot be addressed by scaling the action model alone. Robot datasets [1, 19, 45, 60, 49, 17] collected from different sources use inconsistent observation formats, action spaces, embodiments, and task descriptions, which prevents them from providing coherent supervision without careful alignment. Even after data alignment, policies trained primarily on successful demonstrations receive little guidance for recovering from the error states induced by their own closed-loop behavior. Moreover, a reactive policy conditioned only on the current observation cannot preserve task information once it leaves the observation window. A practical desktop-manipulation system must therefore connect aligned heterogeneous data, robust real-world policy learning, and persistent task-level memory.

We present **Meituan-Robotics-0 (MR0)**, a desktop-manipulation framework built around a large-scale data foundation, a generalist VLA policy, and an event-driven embodied agent, as summarized in Figure 1. The data foundation combines 147 million vision–language samples with 15,060 hours of curated robot trajectories spanning 12 embodiments. The vision–language corpus provides general visual-semantic knowledge together with embodied supervision for grounding, scene understanding, affordance prediction, and task reasoning. The robot corpus comprises three complementary



Figure 1: **Overview of Meituan-Robotics-0.** The framework integrates large-scale multimodal data, three-stage VLA training, an event-driven embodied-agent system, and real-robot deployment.

components: **RoboDesk**, a curated collection of open-source real-robot trajectories totaling 3,235 hours; 7,530 hours of licensed real-robot data spanning seven robot configurations; and 4,295 hours of open simulation data. The complete corpus is processed through temporal and visual filtering, kinematic-consistency checks, cross-embodiment action canonicalization, and hierarchical task alignment. RoboDesk and the simulation data broaden task, environment, and embodiment coverage, while the licensed real-robot data provide task-focused manipulation experience.

Building on this aligned data foundation, the MR0 policy follows a three-stage training recipe that separates the acquisition of broad action priors from continuous-control learning and real-world adaptation. A shared multimodal backbone is first paired with a discrete action head and pre-trained jointly on vision-language data and tokenized robot actions. Continuous mid-training then introduces an MMDiT [46] action head and optimizes flow matching over a unified end-effector representation, transferring the pre-trained features to fine-grained control across single-arm and bimanual embodiments. Real-world post-training [55, 22, 23] starts with supervised finetuning on successful demonstrations. HG-DAgger [18] adds human corrections from policy-induced error states, while RECAP (RL with Experience and Corrections via Advantage-conditioned Policies) [2] derives advantage labels from outcome-labeled autonomous rollouts. We further introduce Rollback, a collection mechanism that returns to a historical anchor to collect a human replacement while retaining the original policy segment. These paired segments supply direct positive and negative labels to CFG-DAgger and remove the need for RECAP's dedicated autonomous rollout collection, value-model training, and advantage labeling. At deployment, asynchronous action-chunk merging integrates new predictions with commands already being executed, maintaining smooth control without blocking on policy inference.

The VLA policy does not explicitly maintain task state across subtasks. MR0 therefore adds an event-driven embodied agent [61, 25, 16] to retain relevant context, track task progress, and revise plans as conditions change. A Monitor summarizes short visual windows into task-relevant events, and a Memory Manager consolidates these events into a persistent record of task progress and environmental conditions. The Planner updates the active subtask only when a new event changes task progress, reveals a failure, or alters whether the current objective is feasible. Motor execution consequently remains with the VLA policy, while the Planner is invoked only when task-level reassessment is needed. This separation enables the system to retrieve information acquired in earlier tasks, suspend an objective whose prerequisites are temporarily unavailable, and resume it after the required conditions are restored.

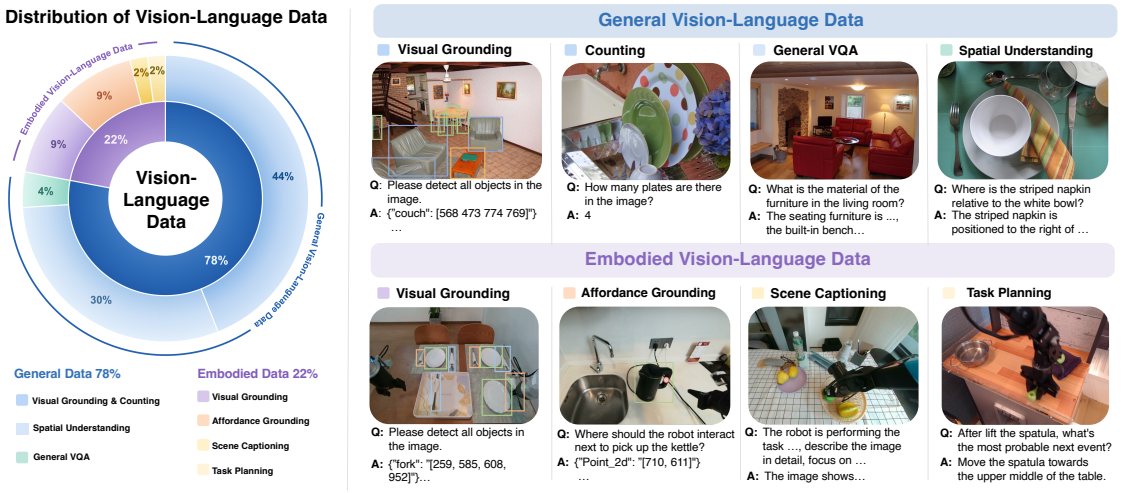


Figure 2: **Vision-language dataset.** The dataset contains 147 million samples, comprising 78% general vision-language data and 22% embodied vision-language data. Representative examples of the major data types are shown on the right.

We evaluate the policy and agent under protocols matched to their respective roles. As a standalone policy, MR0 achieves 93.4% average success on RoboTwin 2.0 [32]. On RoboDojo [10], it ranks fifth in the VLA track despite using a relatively modest training dataset, demonstrating balanced manipulation capabilities and highlighting the data efficiency of its training recipe. Under HG-DAgger post-training with identical data and settings for both models, MR0 improves success rate and progress score on letter-block placement and garment folding compared with $\pi_{0.5}$ [5]. Human-intervention data improve both backbones, showing the importance of supervision in policy-induced states for closed-loop recovery. On Ethernet insertion, CFG-DAgger achieves 70% success and a progress score of 0.90, outperforming HG-DAgger and RECAP using direct supervision from Rollback collection. Separately, qualitative agent deployments demonstrate cross-task event retrieval and conditional task resumption. Together, these evaluations show how aligned data, continuous policy learning, real-world post-training, and event-level memory address complementary requirements of general-purpose desktop manipulation.

2 Data Curation

To equip MR0 with both broad visual-semantic understanding and diverse cross-embodiment manipulation experience, we independently curate two complementary data resources: 147 million vision–language samples and 15,060 hours of robot trajectories spanning 12 embodiments. The vision–language corpus provides supervision for general perception and embodied reasoning. Constructed through a separate pipeline, the robot corpus combines RoboDesk, licensed real-robot data, and open simulation data for action learning across heterogeneous hardware. We describe the two resources in Secs. 2.1 and 2.2, respectively.

Table 1: **Ablation of embodied vision–language supervision in the CALVIN ABC→D setting.** Bold indicates the highest Avg. Len. within each capability family.

Capability family	Training setting	Avg. Len.	Task 1	Task 2	Task 3	Task 4	Task 5
Baseline	–	3.832	91.0%	83.0%	76.2%	70.2%	62.8%
Spatial localization	+ 2D Grounding	3.941	91.3%	83.7%	78.1%	73.2%	67.8%
	+ 2D Pointing	3.806	90.8%	81.8%	74.8%	69.9%	63.3%
	+ 3D Grounding	**4.037**	92.4%	85.3%	79.6%	75.7%	70.7%
Action-related understanding	+ 2D Target-Affordance Grounding	**4.102**	93.3%	87.9%	82.5%	76.4%	70.1%
	+ 3D Target-Affordance Grounding	4.098	93.7%	89.9%	81.2%	73.9%	71.1%
	+ 2D Trajectory	3.987	92.0%	85.0%	79.2%	74.0%	68.5%
Semantic and task reasoning	+ Subtask Prediction	3.851	90.0%	82.2%	76.2%	71.7%	65.0%
	+ Scene Captioning	**4.082**	93.1%	87.1%	81.4%	76.2%	70.4%

2.1 Vision-Language Data

Visual reasoning for manipulation requires both broad perceptual knowledge and embodied understanding grounded in robot interaction. As shown in Figure 2, we organize the 147M vision-language corpus into two complementary subsets. General-purpose data account for 78% and provide supervision for visual question answering, spatial understanding, visual grounding, and counting. The remaining 22% consist of embodied data for scene captioning, task planning, visual grounding, and affordance grounding. Affordance grounding jointly localizes the manipulated object with a bounding box and identifies a 2D interaction point at which the robot should act.

For the embodied VQA component, we augment the collected open-source datasets with a large-scale in-house corpus derived from real-robot trajectories. To determine which supervision types should be prioritized in this corpus, we conduct controlled CALVIN ABC-to-D ablations over eight candidate tasks and use the results to guide subsequent large-scale data construction, as shown in Table 1. The candidates span three capability families: spatial localization, action-related understanding, and semantic and task reasoning. All variants use the same StarVLA [41] configuration, are trained on environments A, B, and C, and are evaluated on environment D. Table 1 reports success rates for completing sequences of 1 to 5 consecutive subtasks, along with the average completed sequence length. The results support including 2D target-affordance grounding and scene captioning as the action-related and semantic supervision types, respectively. For spatial localization, we choose 2D grounding because it avoids costly geometric annotations and scales more readily than 3D grounding. Taken together, the empirical results and practical annotation considerations lead us to focus our in-house data expansion on three types: 2D target-affordance grounding, scene captioning, and 2D grounding.

To ensure consistent and scalable data construction, we develop task-specific annotation pipelines for the selected supervision types. For 2D grounding, we apply Set-of-Mark prompting [52]: the Segment Anything Model (SAM) [20] proposes and marks candidate regions, and a vision-language model assigns an object identity to each region to form grounding question–answer pairs. For 2D affordance grounding, we obtain the target bounding box and interaction point from the key interaction frame of each action phase and propagate both annotations to earlier frames within that phase. For scene captioning, a vision-language model conditions on the robot observation and task context to describe task-relevant objects, spatial relations, gripper state, and execution progress.

2.2 Robotic Data

To provide diverse action supervision for robot control, we assemble 15,060 hours of tabletop trajectories spanning 12 embodiments. We restrict the corpus to stationary-base manipulation and exclude segments containing base motion. As summarized in Figure 3, the corpus consists of RoboDesk (3,235 hours; 21.5%), licensed real-robot data (7,530 hours; 50.0%), and open simulation data (4,295 hours; 28.5%). RoboDesk denotes the six open-source real-world datasets retained after curation, which are listed together with the simulation sources in Table 2.

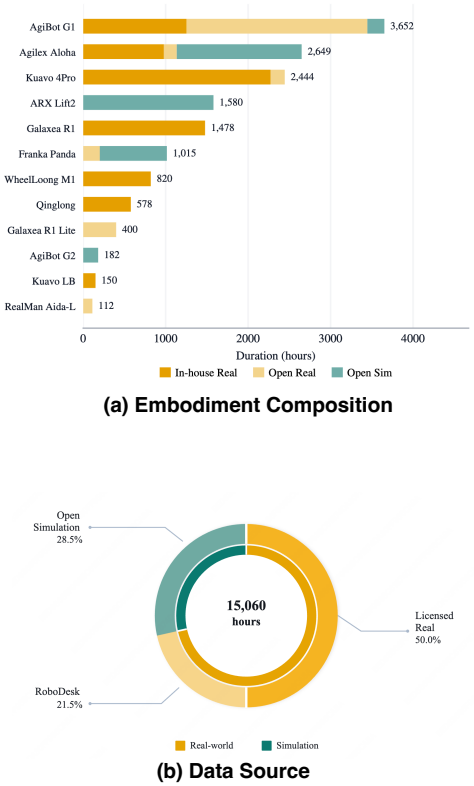


Figure 3: **Composition of the robot corpus.** The 15,060-hour corpus spans 12 embodiments and comprises RoboDesk, a curated collection of open-source real-robot trajectories totaling 3,235 hours (21.5%); 7,530 hours (50.0%) of licensed real-robot data; and 4,295 hours (28.5%) of open simulation data.

2.2.1 Data Curation Pipeline.

Raw trajectories contain idle boundaries, inconsistent kinematic records, visual defects, embodiment-specific action conventions, and uneven language annotations. We address these issues with three filtering stages (S1–S3) followed by two alignment stages (A1–A2). S1 removes static prefixes and suffixes, S2 rejects inconsistent joint–EEF recordings, and S3 removes corrupted visual segments. A1 canonicalizes actions across embodiments, while A2 aligns multi-granularity task annotations with the retained trajectories. Figure 4 summarizes the pipeline.

Static-boundary filtering. Teleoperated recordings often begin before manipulation and continue through task completion or reset, producing long intervals of near-zero motion at episode boundaries. Let $q_t \in \mathbb{R}^J$ be the recorded joint vector and $\Delta q_t^{(j)} = q_t^{(j)} - q_{t-1}^{(j)}$. A time index is considered static only when every joint remains below a motion threshold for L consecutive steps:

$$\text{Static}(t) \iff \left|\Delta q_{t+\ell}^{(j)}\right| < \tau_{\text{stat}}, \quad \forall j \in \{1,\dots,J\}, \quad \forall \ell \in \{0,\dots,L-1\}. \tag{1}$$

The persistence window prevents brief low-speed motions from being classified as idle. We trim only static runs connected to an episode boundary, preserving deliberate pauses during task execution. The corresponding video, state, and action intervals are removed jointly to maintain temporal synchronization.

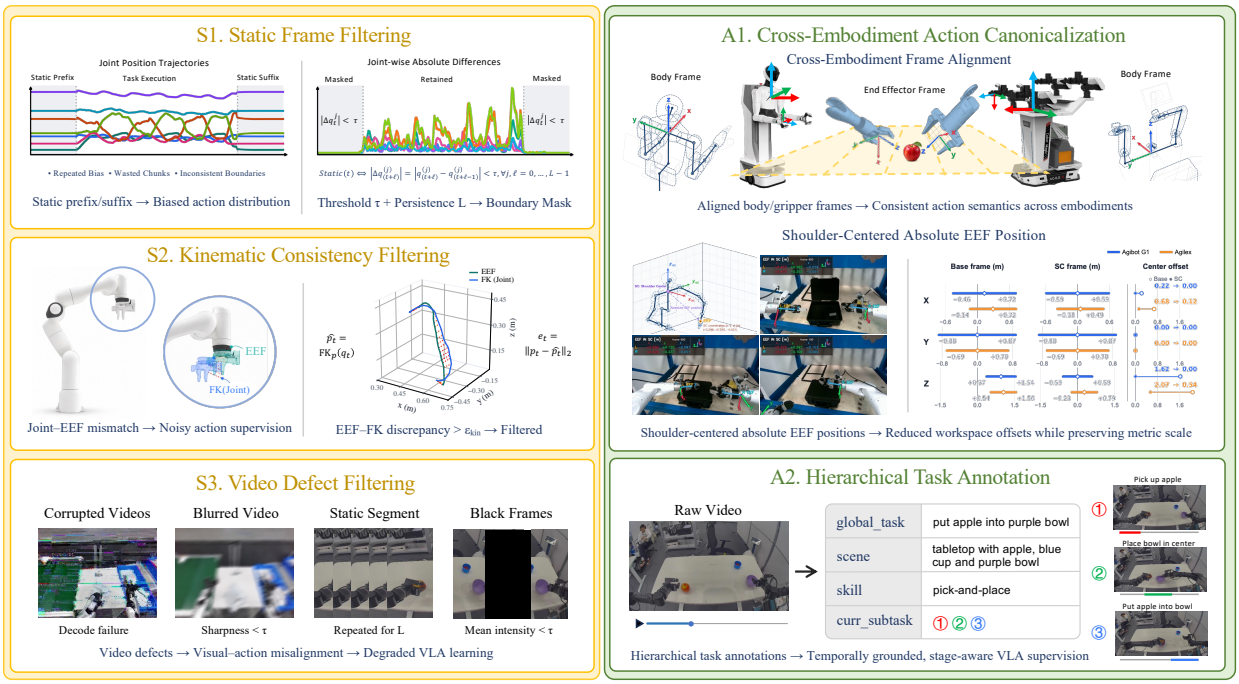


Figure 4: **Robot trajectory curation and alignment pipeline.** S1–S3 remove static boundaries, kinematic inconsistencies, and video defects; A1 canonicalizes actions across embodiments, and A2 aligns multi-granularity task annotations with the retained trajectories.

Kinematic-consistency filtering. For each embodiment, we reconstruct the end-effector position from the recorded joints using the corresponding URDF and compare it with the logged EEF position in a common frame. Samples exceeding an embodiment-specific error threshold are masked, and trajectories with persistent violations are discarded.

Video-defect filtering. We detect decoding failures, blur, frozen segments, and black frames. Corrupted intervals are removed from all synchronized streams, while episodes with persistent defects are discarded. Detection thresholds are calibrated separately by embodiment and data source.

Table 2: Open-source datasets retained after curation.

Data source	Hours	Share
RoboDesk (Open real-world): 3,235 h (21.5%)		
AgiBotWorld-Beta [1]	2,174	14.4%
Galaxea Open-World [17]	333	2.2%
RoboCOIN [49]	311	2.1%
LET [21]	172	1.1%
DROID [19]	164	1.1%
RoboMIND [48]	81	0.5%
Open simulation: 4,295 h (28.5%)		
InternData-A1 [45]	3,967	26.3%
Genie Sim 3.0 [57]	213	1.4%
RoboTwin 2.0 [32]	115	0.8%
Open-source total: 7,530 h (50%)		

Cross-embodiment action canonicalization. To make action channels physically consistent across embodiments, we define common right-handed body and gripper frames. The body axes point forward, left, and up. At the open-gripper zero pose, $+z$ points from the mounting flange to the gripper tip, $+y$ from the right finger to the left, and $+x$ completes the frame. EEF poses are expressed in a canonical shoulder-centered frame, denoted SC. The frame is shared by both arms of a bimanual robot; for a single-arm embodiment without an explicit shoulder center, the arm-root frame is used instead. For arm i, the action contains the absolute EEF position ${}^{\mathrm{SC}}p^{(i)}_{\mathrm{EEF},t} \in \mathbb{R}^3$, a continuous 6D orientation representation $r^{(i)}_t \in \mathbb{R}^6$ [63], and normalized gripper closure $g^{(i)}_t \in [0,1]$, where 0 and 1 denote fully open and fully closed, respectively:

$$a^{(i)}_t = \left[\left({}^{\mathrm{SC}}p^{(i)}_{\mathrm{EEF},t}\right), \left(r^{(i)}_t\right), g^{(i)}_t\right]^\top \in \mathbb{R}^{10}, \qquad a_t = \left[\left(a^{(\mathrm{L})}_t\right), \left(a^{(\mathrm{R})}_t\right)\right]^\top \in \mathbb{R}^{20}. \tag{2}$$

Single-arm samples use the 10-dimensional form. Let B be the source body frame, ${}^{\mathrm{SC}}T_{\mathrm{B}}$ the embodiment-specific rigid transform from B to SC, and ${}^{\mathrm{B}}T^{(i)}_{\mathrm{EEF},t}$ the source EEF target pose. Canonicalization is

$${}^{\mathrm{SC}}T^{(i)}_{\mathrm{EEF},t} = {}^{\mathrm{SC}}T_{\mathrm{B}}\,{}^{\mathrm{B}}T^{(i)}_{\mathrm{EEF},t}, \qquad r^{(i)}_t = \rho_{6\mathrm{D}}\left({}^{\mathrm{SC}}R^{(i)}_{\mathrm{EEF},t}\right). \tag{3}$$

The rotation and translation blocks of ${}^{\mathrm{SC}}T^{(i)}_{\mathrm{EEF},t}$ provide the canonical orientation and metric absolute position target, respectively; the translation is not a temporal displacement.

Hierarchical task annotation. Each episode receives an overall goal in `global_task`, while `curr_subtask` labels the interval in which each execution stage is active. The auxiliary `scene` field describes task-relevant objects and their spatial arrangement, and `skill` names the manipulation primitive. The resulting corpus pairs synchronized observations and canonical EEF actions with episode-level goals, time-aligned subtask labels, and scene and skill context, supporting mixed-granularity instruction conditioning during training.

3 Meituan-Robotics-0

Meituan-Robotics-0 aims to transfer broad priors learned from heterogeneous multimodal data to precise and robust real-robot control. This transition requires the model to preserve its visual-semantic representations while adapting its action interface and supervision to continuous cross-embodiment control and policy-induced error states. We first describe the policy architecture in Section 3.1 and then present the three-stage training pipeline in Section 3.2: discrete multimodal pre-training, continuous cross-embodiment mid-training, and real-world policy optimization.

3.1 Policy Architecture

MR0 formulates low-level desktop manipulation as reactive action-chunk prediction from the current observation. At this level, the policy maintains no temporal history and conditions only on the inputs available at the current decision step. Given multi-view observations $o_t = \{I_t^{(c)}\}_{c=1}^{C}$, language instruction l, and proprioceptive state s_t at time t, MR0 predicts an H-step action chunk

$$\pi_\theta(a_{t:t+H-1} \mid o_t, l, s_t)\,. \tag{4}$$

Following the common VLA design [6, 33, 36], MR0 realizes this mapping with a shared vision–language backbone and two stage-specific output heads, as summarized in Figure 1. The shared backbone is initialized from Qwen3.5-4B [37] and remains trainable throughout the pipeline. During pre-training, FAST [34] applies a discrete cosine transform along the temporal axis and compresses the resulting coefficients with byte-pair encoding. The encoded action chunks share the backbone vocabulary and are predicted by the same autoregressive token head used for text.

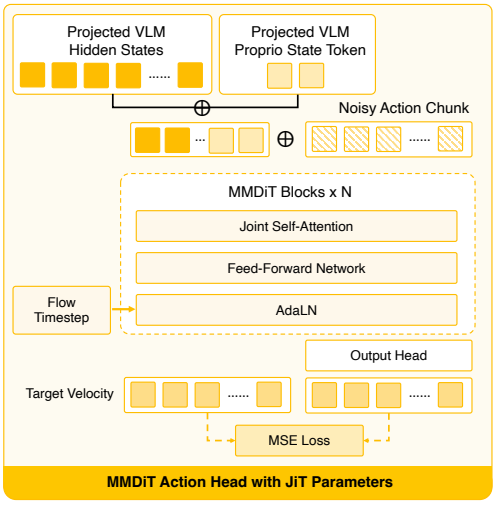


Figure 5: **Continuous action head.** MMDiT denoises action chunks through flow matching.

For mid-training and all downstream stages, MR0 replaces the discrete interface with a continuous MMDiT action head [13, 6]. As illustrated in Figure 5, the head jointly processes an observation stream containing projected backbone features and a proprioceptive-state token, and an action stream containing a noisy action chunk. Joint self-attention exchanges information between the two streams, while the flow timestep modulates block normalization through AdaLN. The state token remains on the observation side, providing condition-side proprioceptive context throughout action generation.

3.2 Progressive Training

MR0 is trained progressively to separate three sources of difficulty: heterogeneous multimodal learning, continuous cross-embodiment control, and recovery from policy-induced errors.

3.2.1 Discrete Multimodal Pre-Training

The shared token interface allows robot trajectories and vision–language samples to use the same autoregressive objective without a separate action decoder. FAST action tokens occupy a dedicated vocabulary block whose embeddings are initialized with the mean of the original embedding table. The remaining backbone parameters are inherited and trained end-to-end, and all camera views share the native vision encoder and aspect-preserving smart-resize policy.

Each optimization step draws one batch from the VLA action pool and one from the vision–language pool. The joint objective is

$$\mathcal{L}_{\mathrm{pre}} = \lambda_{\mathrm{vla}}\mathcal{L}_{\mathrm{action}} + \lambda_{\mathrm{vlm}}\mathcal{L}_{\mathrm{vqa}}, \qquad \lambda_{\mathrm{vla}} = 1.0, \quad \lambda_{\mathrm{vlm}} = 0.1, \tag{5}$$

Table 3: **Pre-training data mixture.** Within-pool ratios are computed from frame counts for the VLA action pool and sample counts for the VLM co-training pool. The reported scales correspond to the effective action pool after stage-specific filtering and therefore differ from the full 15,060-hour corpus summarized in Section 2.2.

Training pool	Data category	Scale	Within-pool ratio
VLA action pool	RoboDesk (open real)	310.7M frames / 3.0k h	21.5%
	Licensed real	746.3M frames / 6.9k h	51.7%
	Synthetic	387.5M frames / 3.6k h	26.8%
	Total	**1,444.5M frames / ∼13.6k h**	**100.0%**
VLM co-training pool	Real-robot grounding	66.7M samples	44.2%
	General VQA / captioning	50.6M samples	33.6%
	Spatial reasoning	11.9M samples	7.9%
	Real-robot affordance	8.2M samples	5.5%
	Simulation VQA	8.2M samples	5.4%
	Task planning / embodied QA	1.8M samples	1.2%
	Real-robot captioning	1.7M samples	1.1%
	Xingzhi VQA	1.6M samples	1.1%
	Total	**150.7M samples from 64 sub-datasets**	**100.0%**

where both terms are next-token cross-entropy over their respective targets. The action loss grounds the backbone in robot behavior, while the VQA loss preserves perceptual and semantic supervision. Table 3 reports the stage-specific mixture. Its 13.6k-hour VLA pool is distinct from the complete 15,060-hour curated inventory in Section 2.2. We train the backbone and extended action-token embeddings end to end for 500,000 steps on 256 GPUs using AdamW with cosine learning-rate decay. The resulting checkpoint initializes mid-training. Because FAST tokenization [34] introduces action-reconstruction error, downstream stages discard the token interface in favor of continuous prediction.

3.2.2 Continuous Cross-Embodiment Mid-Training

The second stage retains the token-pre-trained backbone and initializes the MMDiT action head [13] from scratch. This transfers the visual-semantic and behavioral representation learned during pre-training while replacing quantized action generation with continuous control.

Continuous action objective. We train the MMDiT head with the JiT clean-sample parameterization [26]. Following flow matching [28], a clean chunk a_0 is perturbed into a noisy sample a_τ, from which the head predicts $\hat{a}_0$ rather than velocity directly. Prediction and target are converted to velocity form using a shared clamped denominator d_τ with $t_\epsilon = 0.05$, and the model is optimized by mean squared error.

Unified action representation. To apply the continuous objective across heterogeneous embodiments, MR0 maps each native action to the canonical end-effector convention defined in Section 2.2. Each action chunk has a fixed width $d_{\max} = 20$, corresponding to two 10-dimensional end-effector actions, while a binary mask $m \in \{0,1\}^{d_{\max}}$ identifies the valid channels for each embodiment:

$$\mathcal{L}_{\text{masked}} = \sum_{h=1}^{H} \sum_{j=1}^{d_{\max}} m_j \ell_{h,j} \Big/ H \sum_{j=1}^{d_{\max}} m_j, \tag{6}$$

where $\ell_{h,j}$ is the element-wise JiT loss. Padded channels produce no gradient. For a downstream embodiment with a different action width, only the dimension-dependent projections are reinitialized.

In this stage, we train on 128 GPUs using AdamW with cosine learning-rate decay, applying a lower learning rate to the transferred backbone and a higher rate to the newly initialized action head. The training mixture starts from the data pools summarized in Table 3. We rebalance it by removing a 340-million-frame synthetic sub-corpus, retaining 10% of each of the two largest general-VQA sources, and sampling VLA datasets in proportion to the square root of their frame counts. To expose the model to different levels of linguistic abstraction, we sample the local subtask label or the episode-level goal with equal probability when both are available and otherwise use the available annotation. This `langmix` strategy pairs the same actions with both fine-grained and task-level instructions.

3.2.3 Real-World Policy Optimization

We first fine-tune the mid-training checkpoint π_{mid} on successful demonstrations to obtain an initial task policy. To address policy-induced distribution shift [39], we collect human-in-the-loop (HIL) corrections and compare three post-training algorithms. Human-gated DAgger [18] imitates demonstrations and corrections; RECAP [2] uses trajectory

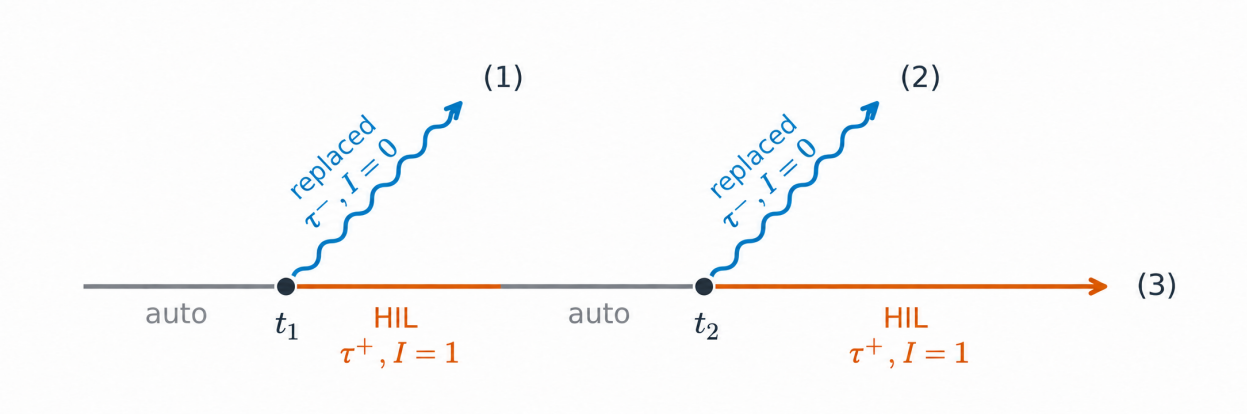


Figure 6: **Rollback data collection.** Dots mark the rollback anchors t_1 and t_2. Wavy blue branches (1) and (2) are replaced policy segments (τ^-, $I = 0$); solid orange HIL segments are their human replacements (τ^+, $I = 1$), with corrected execution continuing toward (3). Gray segments denote other autonomous execution.

outcomes to learn advantage-conditioned behavior; and CFG-DAgger uses positive and negative segments supplied by Rollback. Each collection round adds to the cumulative dataset, and each policy is trained anew from π_{mid}. All three methods execute action chunks through the asynchronous controller in Section 4.1.

Human-gated DAgger. The policy controls the robot until an operator intervenes to correct a stall, error, or unsafe motion. Only human-controlled segments are retained as corrective supervision. Demonstrations cover the task procedure, while HIL corrections add recovery behavior in states encountered by the policy.

Rollback data collection. We introduce Rollback to pair replaced policy segments with human corrections from a shared anchor. An operator pauses execution near failure, selects an earlier recorded state, returns the robot to it using action history, and supplies a corrective continuation. In Figure 6, each anchor (t_1 or t_2) pairs a replaced segment τ^- with its human replacement τ^+. Retaining these suboptimal policy segments is consistent with calls to preserve such data for embodied reward modeling [44]. CFG-DAgger uses the replaced and human segments as negative and positive supervision, respectively, independent of the final episode outcome. RECAP uses complete trajectories reconstructed from the recorded branches for value learning, advantage labeling, and policy training.

RECAP-based policy optimization. RECAP fits a value model to outcome-labeled trajectories and thresholds N-step advantages to assign positive ($I = 1$) or base ($I = \varnothing$) conditions to action chunks. The positive condition may be dropped to train the base branch; chunks assigned $I = \varnothing$ always use the base instruction. Both branches share the masked flow-matching loss. At inference, action-space classifier-free guidance (CFG) combines base and positive predictions generated from the same initial noise. Details are in Section C.2.

CFG-DAgger conditional policy training. CFG-DAgger uses demonstrations and HIL corrections as positive data and replaced segments as negative data. A shared policy learns positive ($I = 1$), negative ($I = 0$), and unconditional ($I = \varnothing$) branches with the same masked flow-matching loss. At inference, CFG combines positive and negative predictions to favor corrective behavior and steer away from replaced actions. Details are in Section C.3.

By obtaining labels directly through Rollback collection, CFG-DAgger requires no retention of autonomous evaluation trajectories for training and avoids value-model training and advantage labeling, while outperforming RECAP in our real-robot evaluation (Section 5.2.2).

4 Embodied Manipulation System

MR0 augments the trained VLA policy with an inference-time system for continuous control and long-horizon task execution. An asynchronous execution layer aligns variable-latency action predictions with robot control, as detailed in Section 4.1. At a higher level, an event-driven agent retains task-relevant state beyond the policy's current observation. As summarized in Figure 8, motor control remains with the VLA policy, and high-level planning is activated only by events that alter task progress or feasibility.

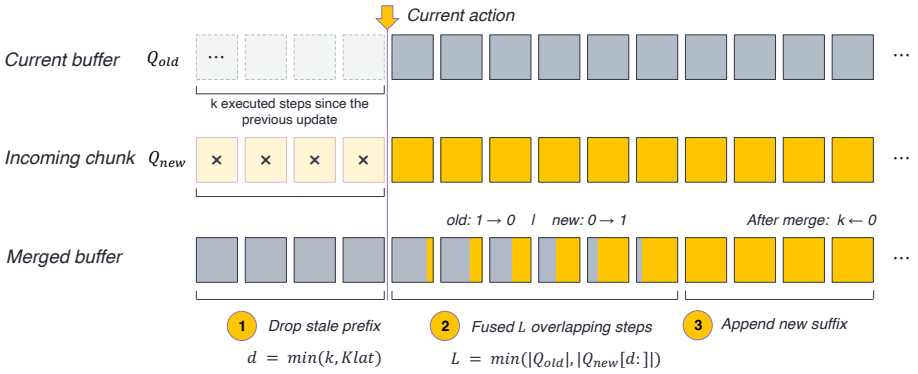


Figure 7: **Asynchronous action-chunk merging.** Predictions made stale during inference are removed from the chunk; the aligned overlap is blended with the residual buffer, and the suffix is appended for continuous execution.

4.1 Asynchronous Policy Execution

Action chunks amortize policy inference over multiple control steps, but inference latency requires coordinating chunk generation with ongoing execution [62, 7, 8]. Our EVA-Client-based stack [51] keeps control continuous by decoupling the two: a background policy thread predicts a chunk from the latest observation at target rate f_π, while the control loop consumes one buffered command at rate f_c. If inference and sequential inverse kinematics exceed the nominal policy period, updates arrive at a lower rate.

This concurrency creates a temporal offset. By the time a new chunk Q_{new} arrives, the controller has consumed k commands since the previous successful update. We denote the unexecuted remainder of the current buffer by Q_{old}. Following the *temporal chunk-wise smoothing* of χ_0 [58], we align the two sequences as shown in Figure 7. We first drop $d = \min(k, K_{\text{lat}})$ leading commands and compute the remaining overlap:

$$\bar{Q}_{\text{new}} = Q_{\text{new}}[d{:}], \qquad L = \min\big(|Q_{\text{old}}|, |\bar{Q}_{\text{new}}|\big)\,. \tag{7}$$

Here K_{lat} is a step-domain cap, set as $K_{\text{lat}} \approx \lceil f_c \hat{\tau}_{\text{e2e}} \rceil$, where $\hat{\tau}_{\text{e2e}}$ is the delay from observation capture to insertion of the processed chunk into the buffer, including communication, policy inference, post-processing, and inverse kinematics.

For each position $i = 0, \ldots, L-1$ in the overlapping region, the updated command is

$$q_i^{\text{fused}} = (1-\alpha_i) q_i^{\text{old}} + \alpha_i \bar{q}_i^{\text{new}}, \qquad \alpha_i = \frac{i}{\max(L-1, 1)}. \tag{8}$$

The unused new suffix fills the future buffer, and k is reset after a successful update. The complete procedure and concrete deployment configuration are given in Section B.

The coupled runtime parameters are f_c, f_π, and K_{lat}. When the inference pipeline keeps pace, roughly f_c/f_π commands are consumed between updates. Increasing f_π refreshes predictions more often, but an overly high update rate repeatedly restarts the transition from the current buffer and makes the physical response sluggish, whereas a low update rate can exhaust the buffer. K_{lat} trades stale-prefix removal against the remaining executable horizon. We use $f_c = 30\,\text{Hz}$, a target $f_\pi = 3\,\text{Hz}$, and $K_{\text{lat}} = 4$ control steps (Table 7).

4.2 Event-Driven Embodied Agent

The chunk merger stabilizes local policy execution but does not preserve information across subtasks or determine when a high-level plan should change. The embodied agent addresses these requirements through explicit task state, persistent event memory, and event-triggered planning.

Task decomposition and execution. Given a user goal g, initial scene context s_0, and skill library $\mathcal{S}$, the Planner first produces an ordered sequence of executable subtasks $Q^{(0)} = \mathcal{P}_{\text{plan}}(g, s_0, \mathcal{S})$. During execution, the agent maintains the task state $X_t = (q_t, R_t)$, where q_t is the active subtask and R_t is the remaining sequence. The active subtask serves as the language instruction to the low-level policy, which generates actions as $\pi_{\text{VLA}}(o_t, q_t)$. It remains active until the agent observes sufficient evidence of completion, failure, or a change in task feasibility. This state representation lets the VLA policy execute continuously while the Planner operates only at task-level transition points.

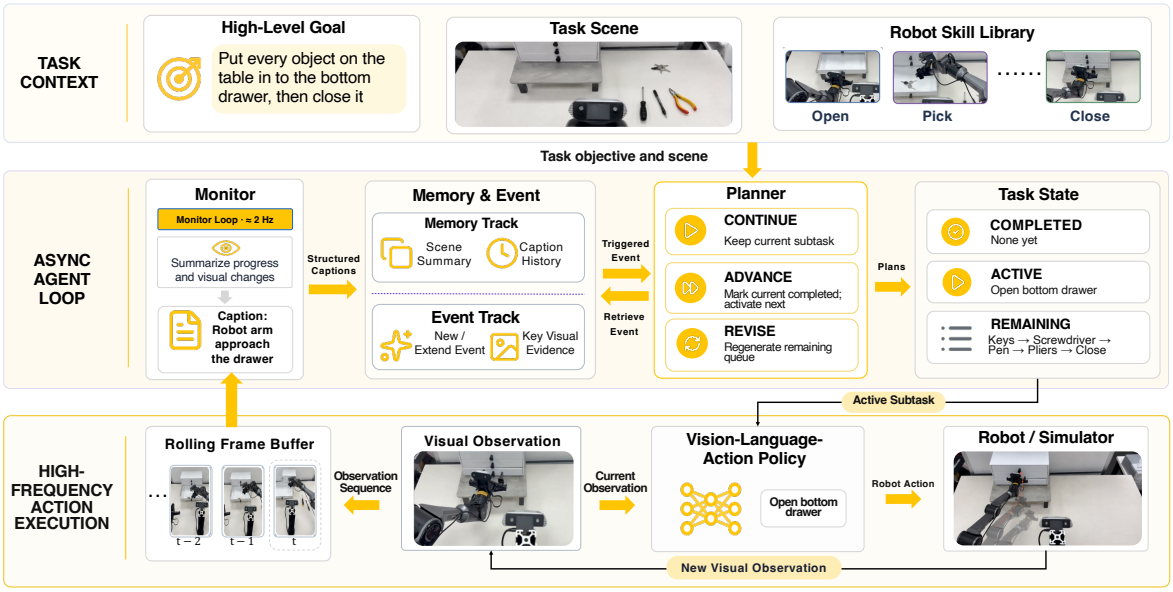


Figure 8: **Event-driven embodied-agent framework.** The asynchronous agent loop converts visual observations into structured captions, consolidates them into events, and triggers the Planner when task-state reassessment is required; the active subtask conditions the high-frequency VLA control loop.

Visual monitoring. The Monitor tracks local execution by mapping a short observation window W_t and the active subtask to a structured caption $C_t = \mathcal{M}(W_t, q_t)$. Conditioning on q_t focuses the description on relevant objects, spatial relations, interactions, and execution progress. The temporal window captures changes such as grasping, placement, or object removal that may be ambiguous in a single frame. The Monitor does not decide whether a subtask has finished or modify X_t; it only produces visually grounded evidence for event construction.

Event construction and memory. Monitor outputs enter a caption buffer in observation-time order. At each memory update, B_t contains the captions that have not yet been processed, and the persistent history $\mathcal{H}_{<t}$ stores ordered event summaries together with their supporting keyframes. The Memory Manager converts the new captions into an event and determines whether that event warrants high-level reassessment:

$$(E_t, z_t) = \mathcal{G}(B_t, \mathcal{H}_{<t}, X_t, g), \qquad z_t \in \{0, 1\}. \tag{9}$$

When the new evidence continues the same interaction, it is merged into the latest memory entry; a distinct state change creates a new entry. For each event, the keyframe set K_t is obtained by uniformly sampling frames from the observation windows associated with its captions. Memory insertion is independent of planner activation: routine observations can be recorded with $z_t = 0$, while subtask completion, execution failure, and changes in task preconditions set $z_t = 1$. This separation preserves information that may become relevant to a later instruction without repeatedly invoking the Planner over an unchanged state.

Event-triggered planning. The Planner performs two related operations. $\mathcal{P}_{\text{plan}}$ constructs or revises a subtask sequence from the goal, scene context, task state, and available skills. When $z_t = 1$, $\mathcal{P}_{\text{decide}}$ instead evaluates the current event, its keyframes, relevant entries retrieved from $\mathcal{H}_{<t}$, the task state X_t, and the user goal g. The resulting decision may preserve the active subtask, advance to the next one, wait for a temporarily unsatisfied precondition, or invoke $\mathcal{P}_{\text{plan}}$ to regenerate the remaining sequence. The Planner judges progress from the observed task state rather than requiring the complete action trajectory. When the event or its visual evidence is inconclusive, it leaves X_t unchanged.

Asynchronous coordination. The Monitor, Memory Manager, Planner, and VLA policy operate at different rates. Multiple Monitor requests may run concurrently, so their outputs can arrive out of order; captions are therefore committed according to observation time before event construction. Planner inference does not pause monitoring or low-level execution, and captions produced during planning remain buffered for the next memory update. Task-state mutations are serialized after each Planner decision, preventing concurrent decisions from modifying the same execution state. The system consequently preserves continuous perception and control while keeping event memory and task transitions temporally consistent.

5 Experiments

5.1 Simulation

RoboTwin2.0. RoboTwin2.0 [32] is a bimanual desktop-manipulation benchmark comprising 50 tasks with a 14-dimensional joint-space action interface. Each task provides 50 demonstrations in the clean-scene split (`demo_clean`) and 500 demonstrations in the randomized split (`demo_randomized`), which varies the background, table clutter, table height, and lighting. We jointly fine-tune all 50 tasks end-to-end from the stage-2 checkpoint using native-resolution inputs. Evaluation covers 100 episodes per task on each split, resulting in 5,000 episodes per split. We evaluate the VLA policy alone, without the agent system or RL post-training.

As shown in Table 4, Meituan-Robotics-0 achieves 93.6% overall success, placing it at the top of the comparison alongside InternVLA-A1.5, while its Clean and Randomized results differ by only 0.5 percentage points. The comparison with ABot-M0 is particularly informative: both methods predict clean actions and optimize a velocity-space loss, yet MR0 achieves 7.8 points higher overall success. Since MR0 adapts a shared stage-2 checkpoint learned from heterogeneous data and embodiments, these results demonstrate ***effective transfer to precise bimanual control and show that action-head parameterization alone does not determine downstream performance.***

Table 4: **RoboTwin 2.0 results.** Success rate (%) over 50 tasks and 100 episodes per task in each split. Overall is the mean of Clean and Randomized; "–" denotes an unreported result. Bold and underlining indicate the best and second-best results.

Method	Clean	Random.	Overall
π_0 [6]	65.9	58.4	62.2
$\pi_{0.5}$ [5]	82.7	76.8	79.8
GR00T N1.6 [33]	47.6	–	–
X-VLA [50]	72.8	72.8	72.8
ABot-M0 [54]	86.1	85.1	85.6
Qwen-VLA-Instruct [36]	86.1	87.2	86.7
LingBot-VLA [47]	86.5	85.3	85.9
Motus [4]	88.7	87.0	87.8
StarVLA [42]	88.7	87.8	88.3
InternVLA-A1 [9]	89.4	89.6	89.5
Being-H0.7 [30]	90.2	89.6	89.9
Fast-WAM [59]	91.9	91.8	91.8
LingBot-VA [24]	92.9	91.5	92.2
InternVLA-A1.5 [35]	93.3	<u>93.0</u>	<u>93.2</u>
GalaxeaVLA (G0.5) [29]	**93.7**	<u>92.8</u>	<u>93.3</u>
Meituan-Robotics-0	<u>93.2</u>	**93.9**	**93.6**

RoboDojo. RoboDojo [10] evaluates generalist manipulation policies under a unified protocol spanning simulation and real-world settings. Its simulation track comprises 42 bimanual tasks across five capability dimensions: Generalization, Precision, Long-Horizon, Memory, and Open-vocabulary instruction following. We evaluate MR0 with an initial-scene reference and latent-action history, without the embodied-agent system or RL-based post-training. Table 5 reports its Score and Success Rate.

To capture both scene changes and recent motion, we retain the first main-camera image of each episode and use a frozen latent-action encoder to extract motion representations from historical image pairs. These representations are averaged within complete one-second intervals, retaining the most recent twenty seconds, and projected into the language-model embedding space with absolute and relative time encodings. Together with the initial-scene reference, current multi-view observations, and task instruction, they condition the existing action generator through the vision-language backbone. During training, we stop gradients through the first-frame visual features and apply consistent color augmentation to the first and current frames from the same camera, while retaining the original action-learning objective.

With approximately only 13.6k hours of embodied training data, MR0 achieves an average Score of 19.68 and an SR of 14.07%. Its Memory and Long-Horizon Scores reach 32.23 and 27.78, respectively, alongside Generalization and Precision Scores of 14.59 and 20.40. Taken together, these results suggest that ***heterogeneous pre-training combined with compact historical context supports competitive performance across multiple manipulation capabilities at a relatively modest data scale***. Open-vocabulary instruction following remains a limitation, with a Score/SR of 3.40/3.00%. The observed tendency to grasp the box rather than align it in `Align_blocks` illustrates the remaining difficulty of translating unfamiliar instructions into appropriate actions.

5.2 Real-Robot Evaluation

We evaluate Meituan-Robotics-0 at two complementary levels. The policy evaluation measures manipulation performance and assesses the contributions of HIL data and, for Ethernet insertion, replaced policy segments retained by Rollback. The agent evaluation examines two history-dependent capabilities: retrieving information no longer available in the current observation and resuming an interrupted task when its execution conditions are restored.

5.2.1 Experimental Setup

Tasks and metrics. We evaluate three tasks covering semantic grounding, deformable-object manipulation, and precision bimanual control (Figure 9). We report success rate (SR) and a task-specific progress score (PS):

Table 5: **RoboDojo simulation results.** The top five WAM and top ten VLA entries are selected by average Score within the official categories as of October 26, 2026. Results are reported as Score/SR (%). *Embodied Data* reports disclosed embodied trajectory/video training hours before RoboDojo adaptation, including inherited pre-training where traceable; – denotes an unavailable verified duration. Bold indicates the best Score or SR among the displayed models in each column. Baseline results follow the official leaderboard; MR0 uses the updated LAM history-input results evaluated in this work. Generalization averages the standard and randomized settings.

Model	Embodied Data	Avg. ↑	Generalization ↑	Precision ↑	Long-Horizon ↑	Memory ↑	Open ↑
WAM							
Liber-0 Lite [27]	–	29.24/24.23	25.36/19.11	36.60/31.08	45.50/32.92	35.68/35.44	3.06/2.58
InternW0-Δ [31]	∼23.1k h[†]	30.77/23.91	30.09/22.78	31.98/23.25	46.29/29.33	34.67/34.00	10.84/10.17
Awomo-0.5 [3]	–	35.34/29.64	30.31/23.84	39.65/33.50	40.36/26.75	42.27/41.22	**24.09/22.92**
VPP2 [15]	–	**39.26/32.26**	**36.38/29.50**	**43.03/35.17**	56.28/37.00	**56.01/55.56**	4.61/4.08
VLA							
SimpleMemVLA [56]	–	12.58/9.27	6.36/3.95	7.42/2.92	14.58/5.50	33.71/33.22	0.85/0.75
KinRT [53]	–	13.02/8.80	14.02/8.61	15.65/9.92	26.40/18.08	4.82/3.56	4.23/3.83
Hy-Embodied-0.5-VLA [60]	>10k h	13.07/8.80	11.78/8.39	13.81/8.00	25.74/14.92	13.37/12.11	0.65/0.58
Xiaomi-Robotics-1 [43]	>110k h[‡]	20.07/13.93	23.54/17.00	26.69/18.83	38.39/23.67	7.81/6.56	3.94/3.58
Meituan-Robotics-0	∼13.6k h	19.68/14.07	14.59/9.33	20.40/12.25	27.78/13.75	32.23/32.00	3.40/3.00
GalaxeaVLA (G0.5) [29]	–	20.23/14.88	18.46/12.83	28.25/20.42	44.12/32.25	8.61/7.33	1.73/1.58
DM0.5 [11]	–	24.90/19.34	15.77/10.95	24.82/16.75	33.70/19.50	47.74/47.44	2.43/2.08
Rex-M1 Preview [14]	>110k h[‡]	33.79/27.44	34.05/26.05	32.82/23.83	46.89/33.00	48.53/48.22	6.67/6.08
Simate-beta [40]	–	33.95/27.96	35.09/27.95	34.35/26.92	**57.84/43.42**	33.33/33.00	9.12/8.50

[†]InternW0-Δ: 23,072.74 processed hours, including 5,633.77 Ego2Robot hours generated from human demonstrations. [‡]Xiaomi's reported stage-wise volumes exceed 100k UMI hours and 10k cross-embodiment hours; their sum is not a verified deduplicated total. Rex-M1 inherits this checkpoint, so its entry is an inherited data-volume lower bound, not a separately disclosed full training total.

- **Letter-block placement.** The robot identifies, reorients, and places lettered blocks into target grooves to spell "MEITUAN". Success requires all seven letters in the correct order and orientation; $\mathrm{PS}_{\mathrm{block}} = k/7$, where k counts consecutive correct placements before the first error.
- **Garment folding.** The robot spreads and folds a garment into a target configuration. Success requires the target fold; $\mathrm{PS}_{\mathrm{fold}} = \mathrm{clip}((300 - T)/210, 0, 1)$ measures execution efficiency, where T is the completion time in seconds and a failed trial receives $T = 300$.
- **Ethernet insertion.** The robot grasps a cable, transfers it between arms, and aligns and inserts the connector into a port. Success requires full insertion; $\mathrm{PS}_{\mathrm{ethernet}} = m/3$, where m counts completed milestones: grasping, handover, and insertion.

Evaluation is autonomous, with matched initial conditions and the same rollout protocol within each comparison. We average SR and PS over evaluation trials.

Post-training data. All tasks use successful teleoperated demonstrations. Letter-block placement and garment folding add corrections collected through standard HIL, whereas Ethernet insertion uses Rollback. Table 6 summarizes the data, with preprocessing described in Section A.

Table 6: **Real-world post-training data.** Demo counts teleoperated demonstrations; HIL / Rollback counts correction episodes collected with each task's mechanism. Training Samples counts valid 50-step action chunks after preprocessing.

Task	Demo	HIL / Rollback	Total	Training Samples
Letter-block placement	500	200	700	∼900K
Garment folding	400	400	800	∼1.1M
Ethernet insertion	200	450	650	∼300K

Compared policies. On letter-block placement and garment folding, we compare MR0 with $\pi_{0.5}$ [5] after SFT on demonstrations and HG-DAgger training on demonstrations and HIL corrections. Both backbones use the same task data, observation and action representations, preprocessing pipeline, and inference settings. On Ethernet insertion, we compare MR0 trained with HG-DAgger, RECAP, and CFG-DAgger using the same demonstrations and Rollback episodes. All three methods are trained from the same mid-training checkpoint, π_{mid}.



Figure 9: **Real-robot evaluation tasks.** Representative stages of (a) letter-block placement, (b) garment folding, and (c) Ethernet insertion.

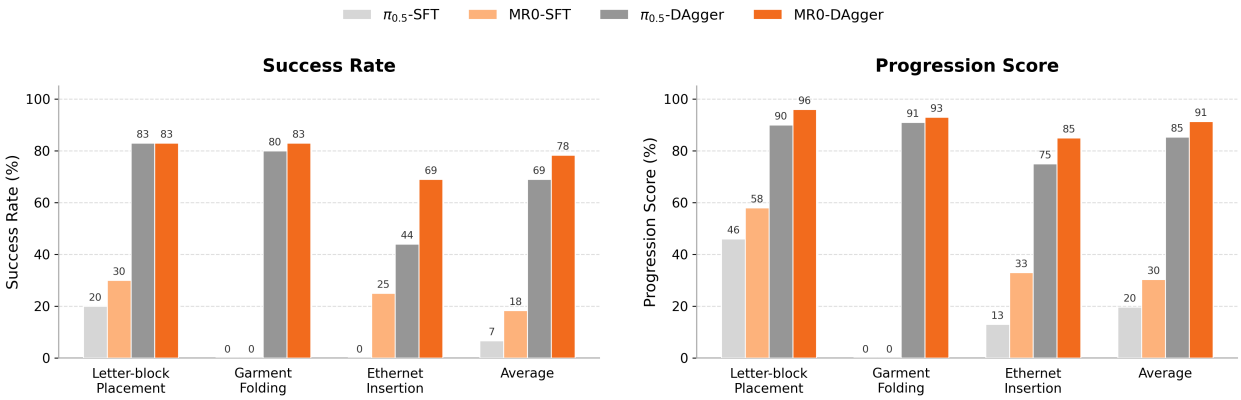


Figure 10: Real-robot SR and PS of $\pi_{0.5}$ and MR0 after SFT and HG-DAgger post-training.

5.2.2 Policy Evaluation

Main comparison. Under matched HG-DAgger post-training, MR0 and $\pi_{0.5}$ achieve the same SR on letter-block placement, while MR0 completes more placements before failure (Figure 10). Garment-folding SR is also similar, but MR0 reduces average completion time from 133 to 117 seconds. The gains are primarily in completion quality and efficiency.

Effect of HIL data. HIL corrections substantially improve both backbones (Figure 10). Neither SFT policy succeeds on garment folding, highlighting the limited coverage of failure and recovery states in successful demonstrations alone. As HIL data increase (Figure 11), letter-block placement and garment folding improve rapidly before plateauing, while Ethernet insertion continues to improve. The gains from additional corrections therefore depend on the task and existing data coverage.

Learning from Rollback pairs. CFG-DAgger achieves higher SR and PS than both baselines across the evaluated data scales (Figure 12), reaching 0.70 SR and 0.90 PS, compared with 0.60 SR and 0.67 PS for RECAP. These results show that *Rollback pairs support effective policy optimization without additional autonomous data collection, value learning, or advantage labeling.*

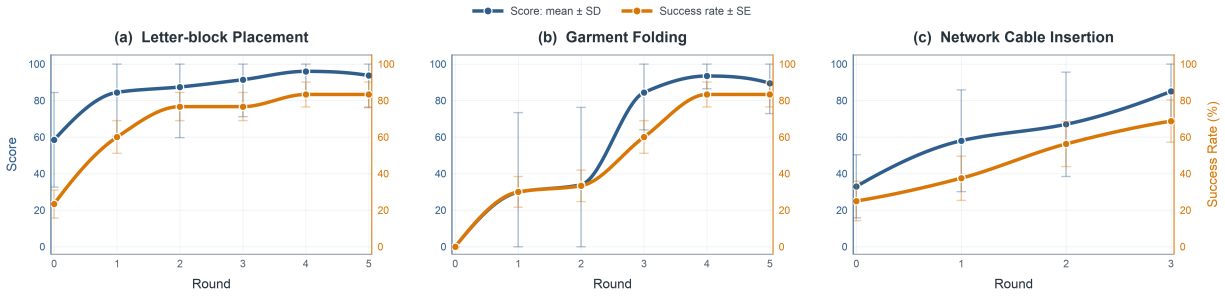


Figure 11: Effect of accumulating HIL data on the three tasks. The initial point uses demonstrations only; subsequent points add accumulated corrections. Each policy is trained independently from the same mid-training checkpoint.

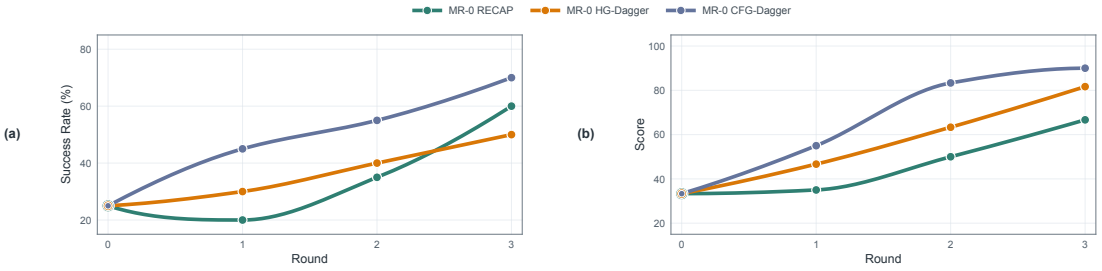


Figure 12: Ethernet insertion performance with increasing Rollback data. Fractions indicate the proportion of the full Rollback set used with a fixed demonstration set; all methods share the same SFT reference.

5.2.3 Embodied Agent Evaluation

The policy experiments measure execution within a single task, but do not test whether the system can recover information that has disappeared from the current observation or preserve an objective across an interruption. We therefore conduct two controlled qualitative deployments: cross-task event retrieval and condition-aware task resumption.

Agent setup. The Monitor, Memory Manager, and Planner use Qwen3.5-35B-A3B and share an SGLang inference pool with four instances. The Monitor runs at 2 Hz over an eight-frame, two-second observation window. Each persistent event stores a semantic summary and four to eight archived keyframes.

Cross-task event retrieval. In Figure 13, the initial instruction is to place the tabletop objects in the bottom drawer and close it. After the key has been placed and the screwdriver subtask has begun, a person moves the key to the middle drawer. The Monitor captures this state change, and the Memory Manager stores the event with its keyframes. Because the active screwdriver subtask remains feasible, execution continues and the initial instruction is completed. When the user later requests the key, neither the closed cabinet nor the empty table reveals its location. The agent retrieves the stored event, identifies the middle drawer as the key's last observed location, and generates a plan to recover it. This case shows that event summaries and archived visual evidence can support a new task when the relevant state is no longer directly observable.

Interruption and condition recovery. In Figure 14, the agent is instructed to place the key in the bottom drawer and close it. A person removes the key after the drawer is opened but before the grasp is completed. The Monitor detects that the target is unavailable, while the Memory Manager records the condition change. The Planner retains the pending placement subtask and enters a waiting state rather than discarding or replanning the objective. When the key returns, a recovery event triggers reassessment against the latest observation, after which the robot places the key and completes the original instruction. This case demonstrates that persistent task state can preserve intent during temporary infeasibility and resume execution when the missing condition is restored.

Together, the two cases exercise complementary forms of temporal context. *Event memory and persistent task state extend planning beyond the current observation by supporting cross-task retrieval and interrupted-task resumption.*

6 Conclusion

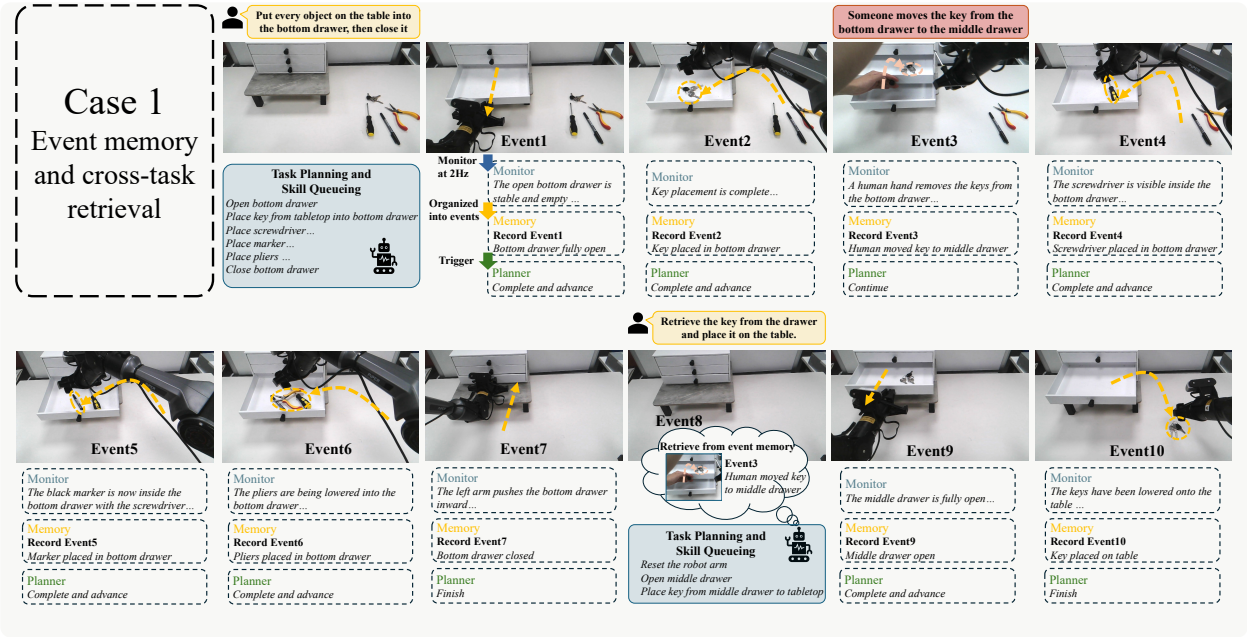


Figure 13: **Event memory and cross-task retrieval.** While screwdriver placement remains active, a person moves the previously placed key from the bottom drawer to the middle drawer. The stored event is later retrieved to locate the key for a new instruction.

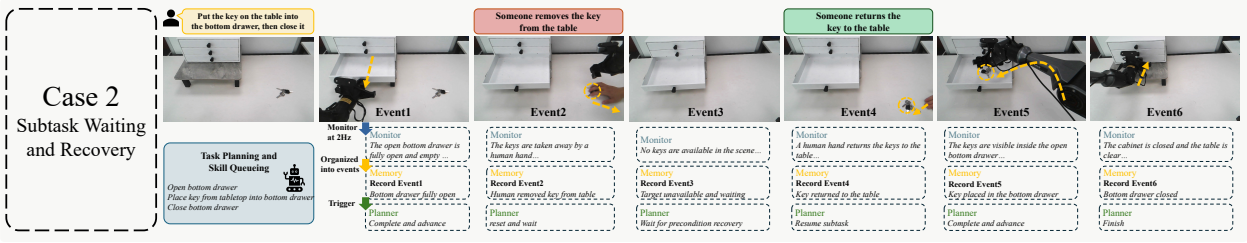


Figure 14: **Subtask waiting and condition recovery.** Removing the key makes the active placement subtask temporarily infeasible. The agent retains the pending objective and resumes execution after the key returns.

7 Contributions

All people are cataloged **alphabetically by first name**. (†) indicates the project leader and (‡) indicates the sponsors.

Contributors:

Caicheng Wang
Chen Zhao
Chengpeng Jin
Duo Xu
Fengjiao Chen
Guanglu Wan‡
Heqing Yang
Jia Wang
Jiacong Zhou
Jie Hu†
Jiyuan Ru
Linjiang Huang
Manyuan Zhang
Ningchu Liu
Shijie Zhou
Si Liu
Shuo Li
Tianyi Zhang
Xi Zhang
Xin Chen
Xuezhi Cao
Xunliang Cai‡
Yanbin Zeng
Yang Yang
Yang Yi
Yanli Tan
Yifu Xu
Yimeng Jia
Yuchen Tang
Yuchen Xie‡
Yuxiang Zheng
Zerwei Chng
Zilong Huang
Zipeng Ye
Ziqin Wang
Ziyuan Qin

A Real-Robot Setup and Data Preparation

Data interface across teleoperation, HIL, and autonomous rollouts. Our bimanual AgileX PiPER setup contains two human-operated leader arms and two corresponding task-executing follower arms. The operator moves the leader arms, and their joint positions are relayed as target commands to the follower arms. Each trajectory records three camera streams from the task workspace, the observed follower-arm configuration in `observations/qpos`, and the robot command in `action`. During teleoperation and human intervention, `action` is constructed from the leader-arm joint positions and sent to the follower arms; during autonomous execution, it is produced by the policy. Thus, leader-arm state is not a policy observation, but it does determine the supervised action target for human-controlled frames. Both control sources use the same stored camera, state, and action interface.

The demonstration dataset $\mathcal{D}_{\text{demo}}$, collected by teleoperation, contains only successful executions. The tasks and data quantities used in our experiments are specified in Section 5.2.

Real-robot observation and action representation. The policy receives synchronized images from three RGB cameras: one head camera and two wrist cameras. Each camera captures images at a native resolution of 640×480. The policy uses a single observation frame and does not include an image history.

Rather than warping images to a fixed square resolution, we apply the Smart Resize operation of Qwen3.5-VL, which bounds the number of visual tokens while preserving the aspect ratio; images are resized to approximately 352×256 before visual encoding, identically during post-training and inference.

The policy state is derived from the observed follower-arm configuration, and the action target from the command sent to the follower arms. Both are converted to the same 20-dimensional end-effector representation. For each arm, this representation contains a 3D Cartesian position, a 6D rotation, and an absolute gripper opening value:

$$\mathbf{x}_t = \left[\mathbf{p}^L, \mathbf{r}^L_{\text{6D}}, g^L, \mathbf{p}^R, \mathbf{r}^R_{\text{6D}}, g^R\right] \in \mathbb{R}^{20}, \qquad \mathbf{x}_t \in \{\mathbf{s}_t, \mathbf{a}_t\}. \tag{10}$$

The policy predicts an action chunk containing $H = 50$ control steps:

$$\mathbf{a}_{t:t+H-1} = [\mathbf{a}_t, \mathbf{a}_{t+1}, \ldots, \mathbf{a}_{t+49}] \in \mathbb{R}^{50\times 20}. \tag{11}$$

Teleoperation, HIL collection, autonomous execution, and policy inference all operate at 30 Hz, so every data source shares the same control frequency.

Static-interval trimming and valid starts. Before constructing post-training samples, we remove sustained intervals in which changes in both observed arm configuration and commanded actions remain below task-specific thresholds. This limits the dominance of hold actions while preserving deliberate slow motion and corrections.

After trimming and source selection, trajectories are split at episode and control-source boundaries. A valid start contains $H = 50$ recorded actions from one retained segment, without tail padding or crossing a takeover boundary. Each valid start contributes one training sample in Table 6. SFT uses demonstrations; HG-DAgger adds human corrections; CFG-DAgger also uses replaced segments; RECAP uses demonstrations and complete trajectories reconstructed from Rollback episodes. All retained SFT and HG-DAgger samples receive equal sampling weight.

Post-training implementation details. Small random crops and rotations augment the head-camera view; color jitter augments all three views. State and action dimensions are normalized using their first and 99th percentiles, with the same training-set statistics used to unnormalize actions during deployment.

B Asynchronous Policy Execution Details

This section gives the complete buffer-update procedure for the asynchronous controller summarized in Section 4.1. The control and inference loops share a locked action buffer and run concurrently.

The async preset leaves `execute_horizon` unset and uses `action_time_scale`=1.0, so the client retains all $H = 50$ returned actions before latency trimming. The two warm-up queries prime the inference and IK pipeline; the action buffer is then cleared and anchored at the current robot state, rather than prefilled with two predicted chunks.

Table 7: Configuration of the asynchronous executor.

Quantity or option	Value	Role
Strategy	`async/linear_overlap`	Background prediction with latency-aware buffer updates.
f_c (`frame_rate`)	30 Hz	Command-consumption rate of the control loop; `publish_rate` is matched to the same value.
f_π (`inference_rate`)	3 Hz	Target background prediction rate.
H	50	Action horizon returned by the policy server.
K_{lat} (`latency_k`)	4	Maximum number of leading commands removed from an incoming chunk.
$m_{\min}$ (buffer default)	8	Minimum old-reference length used for short-buffer padding.
`setup_warmup_chunks`	2	Synchronous startup queries before the background loop begins.

C Real-World Post-Training Details

Supervised fine-tuning on successful demonstrations initializes data collection. HG-DAgger, RECAP, and CFG-DAgger then use the accumulated data in different ways, with each policy trained anew from π_{mid}. In the Ethernet insertion comparison, all three methods use the same demonstrations and Rollback episodes: HG-DAgger retains human corrections, CFG-DAgger also retains replaced segments, and RECAP reconstructs complete trajectories for value learning, advantage labeling, and policy training. Episodes are stored once in acquisition order, with segment annotations defining the branches. All methods use the interface in Section A.

C.1 HG-DAgger Human Intervention

Let π_0 be the SFT policy trained on demonstrations $\mathcal{D}_{\text{demo}}$. At collection round k, a binary gate g_t selects the current policy or human control:

$$a_t^{\text{exec}} = (1 - g_t)\, \pi_{k-1}(o_t, \ell) + g_t\, \pi_H(o_t), \tag{12}$$

where $g_t = 1$ selects the human policy π_H. The operator intervenes when execution stalls, errs, or becomes unsafe, and returns control after recovery. Only human-controlled samples from successful episodes are retained, whether collected through standard HIL or Rollback:

$$\mathcal{D}_{\text{HIL}}^{(k)} = \left\{ (o_t, a_t^H) \;\middle|\; g_t = 1 \right\}. \tag{13}$$

After round k, the cumulative supervised dataset is

$$\mathcal{D}_{\text{DAgger}}^{(k)} = \mathcal{D}_{\text{demo}} \cup \bigcup_{i=1}^{k} \mathcal{D}_{\text{HIL}}^{(i)}. \tag{14}$$

HG-DAgger changes data collection and sample selection but retains the masked flow-matching action loss.

C.2 RECAP Policy Optimization

Trajectory reconstruction. For each Rollback episode, we reconstruct complete paths by joining the recorded prefix to each replaced branch or corrected continuation. In Figure 6, paths ending at (1) and (2) supply failed trajectories, while the path to (3) supplies a successful trajectory. RECAP uses these trajectories for value learning, advantage labeling, and policy training; no separate autonomous collection is required for this comparison. Let $\mathcal{T}_{\text{RB}}^{(k)}$ denote the trajectories reconstructed from collection round k. The cumulative dataset is

$$\mathcal{D}_{\text{RL}}^{(k)} = \mathcal{D}_{\text{demo}} \cup \bigcup_{i=1}^{k} \mathcal{T}_{\text{RB}}^{(i)}. \tag{15}$$

Algorithm 1 Latency-Aware Asynchronous Policy Execution

Require: Action horizon H, control rate f_c, target inference rate f_π, latency cap K_{lat}, minimum old-reference length $m_{\min}$

Shared state: buffer $Q \leftarrow \emptyset$, counter $k \leftarrow 0$, and $q_{\text{last}} \leftarrow q_{\text{current}}$
Each pop and each complete buffer update are protected by the same lock.
Run the following control and inference loops concurrently.
Control loop (at f_c):
while the system is running **do**
  if $Q \neq \emptyset$ **then**
    $q \leftarrow \text{POPFRONT}(Q)$; $k \leftarrow k + 1$
    if $Q = \emptyset$ **then**
      $q_{\text{last}} \leftarrow q$
    end if
    $\text{PUBLISH}(q)$
  end if
end while
Inference loop (at target f_π):
while the system is running **do**
  $Q_{\text{new}} \leftarrow \text{IK}(\text{PREDICT}(\text{OBSERVE}()))$ ▷ $|Q_{\text{new}}| = H$
  if $Q_{\text{new}} = \emptyset$ **then**
    continue
  end if
  $d \leftarrow \min(k, K_{\text{lat}})$
  if $d \geq |Q_{\text{new}}|$ **then**
    continue ▷ Q and k remain unchanged
  end if
  $\bar{Q}_{\text{new}} \leftarrow Q_{\text{new}}[d{:}]$
  if $Q = \emptyset \wedge q_{\text{last}} \neq \varnothing$ **then**
    $Q_{\text{old}} \leftarrow [q_{\text{last}}]^{m_{\min}}$; $q_{\text{last}} \leftarrow \varnothing$
  else if $Q = \emptyset$ **then**
    $Q \leftarrow \bar{Q}_{\text{new}}$; $k \leftarrow 0$
    continue
  else
    $Q_{\text{old}} \leftarrow Q$
    if $|Q_{\text{old}}| < m_{\min}$ **then**
      Extend Q_{old} to $m_{\min}$ by repeating its terminal command
    end if
  end if
  $L \leftarrow \min(|Q_{\text{old}}|, |\bar{Q}_{\text{new}}|)$
  for $i = 0$ to $L - 1$ **do**
    $\alpha_i \leftarrow i / \max(L - 1, 1)$
    $q_i^{\text{fused}} \leftarrow (1 - \alpha_i) q_i^{\text{old}} + \alpha_i \bar{q}_i^{\text{new}}$ ▷ Linear transition
    Overwrite the gripper entries of q_i^{fused} with those of $\bar{q}_i^{\text{new}}$
  end for
  $Q \leftarrow [q_0^{\text{fused}}, \ldots, q_{L-1}^{\text{fused}}] \,\|\, \bar{Q}_{\text{new}}[L{:}]$
  $k \leftarrow 0$
end while

Outcome rewards and value targets. Episode-level outcomes are converted into per-timestep rewards:

$$r_t = \begin{cases} 0, & t = T \text{ and the task succeeds,} \\ -T_{\max}, & t = T \text{ and the task fails,} \\ -1, & \text{otherwise,} \end{cases} \qquad R_t = \sum_{t'=t}^{T} r_{t'}, \tag{16}$$

where T is the terminal timestep and $T_{\max}$ is the maximum episode length. Nonterminal steps incur unit cost, and failure adds a terminal penalty of $T_{\max}$. Normalizing by $D = 2T_{\max}$ gives the clipped value target

Algorithm 2 HG-DAgger real-world post-training

Require: Mid-training checkpoint π_{mid}, demonstration dataset $\mathcal{D}_{\text{demo}}$, rounds K

$\mathcal{D}^{(0)}_{\text{DAgger}} \leftarrow \mathcal{D}_{\text{demo}}$
Train π_0 from π_{mid} on $\mathcal{D}^{(0)}_{\text{DAgger}}$
for $k = 1$ to K **do**
  Deploy π_{k-1} under operator gating (Equation (12)); collect $\mathcal{D}^{(k)}_{\text{HIL}}$
  $\mathcal{D}^{(k)}_{\text{DAgger}} \leftarrow \mathcal{D}^{(k-1)}_{\text{DAgger}} \cup \mathcal{D}^{(k)}_{\text{HIL}}$
  Train π_k from π_{mid} on $\mathcal{D}^{(k)}_{\text{DAgger}}$
end for

Ensure: Final task policy π_K

Algorithm 3 RECAP policy optimization with Rollback trajectories

Require: π_{mid}, value initialization V_{init}, demonstrations $\mathcal{D}_{\text{demo}}$, Rollback batches $1, \ldots, K$

$\mathcal{D}^{(0)}_{\text{RL}} \leftarrow \mathcal{D}_{\text{demo}}$
for $k = 1$ to K **do**
  Reconstruct trajectories and outcomes $\mathcal{T}^{(k)}_{\text{RB}}$ from Rollback batch k
  $\mathcal{D}^{(k)}_{\text{RL}} \leftarrow \mathcal{D}^{(k-1)}_{\text{RL}} \cup \mathcal{T}^{(k)}_{\text{RB}}$
  Fit V_k from V_{init} on $\mathcal{D}^{(k)}_{\text{RL}}$ using outcome-derived targets
  Assign conditioning states $I_t \in \{1, \varnothing\}$ to all valid starts, including demonstrations and HIL, using Equation (24)
  Train π^{RL}_k from π_{mid} on $\mathcal{D}^{(k)}_{\text{RL}}$ with advantage conditioning
end for

Ensure: Final task policy π^{RL}_K

$$y_t = \text{clip}\left(\frac{R_t}{D}, -1, 0\right) = \begin{cases} \text{clip}\left(-\dfrac{T-t}{D}, -1, 0\right), & \text{success}, \\ \text{clip}\left(-\dfrac{T-t+T_{\max}}{D}, -1, 0\right), & \text{failure}. \end{cases} \tag{17}$$

Targets lie in $[-1, 0]$, with values closer to zero indicating shorter successful completions and the terminal penalty distinguishing failures.

Distributional value prediction. The value network predicts a categorical distribution over $B = 201$ atoms:

$$p_\phi(V \mid o_t, \ell) \in \Delta^B, \qquad z_i = V_{\min} + i\frac{V_{\max} - V_{\min}}{B-1}, \quad i = 0, \ldots, B-1, \tag{18}$$

where $V_{\min} = -1$, $V_{\max} = 0$, and the atom spacing is $\Delta z = 0.005$. We project a scalar target onto its two neighboring atoms using

$$b_t = \text{clip}\left(\frac{y_t - V_{\min}}{\Delta z}, 0, B-1\right), \qquad l = \lfloor b_t \rfloor, \qquad u = \lceil b_t \rceil. \tag{19}$$

When $l < u$, the target masses are $q_l = u - b_t$ and $q_u = b_t - l$. When $l = u$, we set $q_l = 1$; all other entries are zero. The value loss and continuous prediction are

$$\mathcal{L}_V = -\sum_{i=0}^{B-1} q_i \log p_\phi(V = z_i \mid o_t, \ell), \qquad V(o_t, \ell) = \sum_{i=0}^{B-1} p_\phi(V = z_i \mid o_t, \ell) z_i. \tag{20}$$

Value-model architecture. Following RLinf's RECAP implementation [38], a SigLIP2-So400m/14 encoder maps each of the three RGB views to 256 tokens projected to 640 dimensions. An 18-layer Gemma 3 270M backbone encodes these visual tokens together with the instruction and discretized robot state.

A separate 18-layer value expert with hidden dimension 256 updates a learnable [CLS] token. It attends to the vision-language prefix through shared attention keys and values, while the prefix cannot attend back to it. A linear head predicts the value distribution:

$$h_t^{\mathrm{CLS}} \in \mathbb{R}^{256} \longrightarrow \mathrm{logits}_t = W_V h_t^{\mathrm{CLS}} + b_V \in \mathbb{R}^{201}. \tag{21}$$

The vision encoder and Gemma backbone use pretrained weights, the value expert and head are randomly initialized, and the [CLS] token starts at zero. All components are trained jointly.

N-step advantage labeling. Transition rewards use the same scale as the normalized value targets:

$$\widetilde{r}_t = \begin{cases} y_t - y_{t+1}, & t < T, \\ y_T, & t = T. \end{cases} \tag{22}$$

For a valid start frame with a complete N-step horizon, the advantage is

$$A_t^{(N)} = \sum_{k=0}^{N-1} \widetilde{r}_{t+k} + V(o_{t+N}, \ell) - V(o_t, \ell). \tag{23}$$

If fewer than N steps remain, rewards are accumulated through termination and the bootstrap value beyond the episode is set to zero. Before condition dropout, we assign a conditioning state $I_t \in \{1, \varnothing\}$:

$$I_t = \begin{cases} 1, & A_t^{(N)} \geq \epsilon, \\ \varnothing, & A_t^{(N)} < \epsilon. \end{cases} \tag{24}$$

Here, $N = H = 50$ and ϵ is the 70th percentile of advantages over the aggregated dataset. Each valid action chunk inherits its start-frame conditioning state. This rule applies to all sources, including demonstrations and HIL corrections; their source does not automatically assign a positive condition.

Advantage-conditioned policy training. Before condition dropout, the conditioning state determines the suffix appended to the task instruction:

$$c_t = \begin{cases} \texttt{Advantage: positive}, & I_t = 1, \\ \varnothing, & I_t = \varnothing. \end{cases} \tag{25}$$

For chunks with $I_t = 1$, the positive suffix is omitted with probability $p_{\mathrm{drop}} = 0.3$, so the policy receives only the base instruction. This dropout leaves the advantage-derived label unchanged. Chunks with $I_t = \varnothing$ always use the base instruction. The base branch thus receives both below-threshold chunks and positive chunks after condition dropout. Omitting the condition does not discard the sample: all retained chunks use the masked flow-matching action loss. The value model is trained separately with the cross-entropy in Equation (20).

Action-space classifier-free guidance. At inference, we denoise a base prediction $\mathbf{a}_\varnothing$ ($I = \varnothing$) and a positive-conditioned prediction $\mathbf{a}_+$ ($I = 1$) from the same initial noise, then combine the completed action chunks once in action space:

$$\mathbf{a}_{\mathrm{CFG}} = \mathbf{a}_\varnothing + \beta\,(\mathbf{a}_+ - \mathbf{a}_\varnothing)\,, \qquad \beta = 1.75. \tag{26}$$

The base branch omits only the advantage condition, retaining the observation, state, and task instruction. Guidance is applied once to the completed chunks.

Training configuration. Table 8 summarizes the training configuration.

Table 8: RECAP value learning, advantage labeling, and policy-training hyperparameters.

Parameter	Setting
Value support $[V_{\min}, V_{\max}]$	$[-1, 0]$
Number of value atoms B	201
Value training steps / batch size	6,000 / 256
Value peak learning rate	5×10^{-5}
Value warmup / final learning rate	500 / 1×10^{-6}
Advantage horizon N / action horizon H	50 / 50
Advantage threshold ϵ	Aggregated-data 70th percentile
Condition-dropout rate p_{drop}	0.3
Policy training steps / batch size	30,000 / 128
Policy peak learning rate	1×10^{-5}
Policy warmup / minimum learning rate	10,000 / 5×10^{-7}
Inference guidance scale β	1.75

C.3 CFG-DAgger Conditional Policy Training

Data and labels. In Figure 6, each pair (τ^-, τ^+) shares an anchor but may diverge afterward; no frame-wise alignment is required. Replaced segments carry $I = 0$, while demonstrations and HIL corrections carry $I = 1$:

$$\mathcal{D}^+ = \mathcal{D}_{\text{demo}} \cup \mathcal{D}_{\text{HIL}}, \qquad \mathcal{D}^- = \bigcup_j \tau_j^-, \tag{27}$$

where $\mathcal{D}_{\text{HIL}}$ includes the human replacements τ^+. Labels describe local segments, independent of episode outcomes. If the scene cannot be restored to an anchor, collection falls back to standard HIL and contributes only positive data. These labels require neither value learning nor advantage estimation.

Conditional policy training. A shared policy $\pi_\theta(a_{t:t+H-1} \mid o_t, \ell, s_t, I)$ learns positive ($I = 1$), negative ($I = 0$), and unconditional ($I = \varnothing$) branches using the masked flow-matching loss. Here, $I = 0$ supplies an explicit negative condition, while RECAP uses $I = \varnothing$ for its base instruction.

Sampling and iterative training. Positive samples use $\mathcal{D}^+$, with condition dropout of 0.3 to train the unconditional branch while retaining the task instruction. Negative samples from $\mathcal{D}^-$ always retain their condition. Other autonomous segments, rollback/reset motions, and segments shorter than $H = 50$ are excluded. Valid chunks are sampled without class rebalancing, and each round trains from π_{mid} on the cumulative dataset.

Action-space classifier-free guidance. At inference, we denoise a positive prediction $\mathbf{a}_+$ ($I = 1$) and a negative prediction $\mathbf{a}_-$ ($I = 0$) from the same initial noise, then combine the completed action chunks once in action space:

$$\mathbf{a}_{\text{CFG}} = \mathbf{a}_- + \gamma\,(\mathbf{a}_+ - \mathbf{a}_-)\,. \tag{28}$$

We use $\gamma = 1.25$, extrapolating from the positive prediction away from the negative prediction. The unconditional branch is used only during training.

References

[1] AgiBot-World-Contributors et al. AgiBot World Colosseo: A large-scale manipulation platform for scalable and intelligent embodied systems. *arXiv preprint arXiv:2503.06669*, 2025.

[2] A. Amin, R. Aniceto, A. Balakrishna, et al. $\pi^*_{0.6}$: A vla that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.

[3] Awomo-WAM Team. Awomo-0.5 Technical Report. 2026.

[4] Kaiwen Bi et al. Motus: Motion priors improve vision-language-action models. *arXiv preprint*, 2025.

[5] Kevin Black, Noah Brown, James Darpinian, Karan Dhariwal, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[6] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[7] Kevin Black, Manuel Y. Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems*, volume 38, pages 37596–37620, 2025.

[8] Kevin Black, Allen Z. Ren, Michael Equi, and Sergey Levine. Training-time action conditioning for efficient real-time chunking. *arXiv preprint arXiv:2512.05964*, 2025.

[9] Zhenyu Cai et al. InternVLA-A1: Unifying understanding and action via latent action modeling. *arXiv preprint*, 2026.

[10] Tianxing Chen, Yue Chen, Zixuan Li, Junyuan Tang, Kailun Su, Weijie Wan, Baijun Chen, Haoran Lu, Haowen Yan, Honghao Su, et al. RoboDojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies. *arXiv preprint arXiv:2607.04434*, 2026.

[11] Dexmal. DM0.5: Designed for the Open World, Where Generalization Emerges. https://www.dexmal.com/blog/dm0.5, 2026.

[12] Zi-Han Ding and Ziwei Wang. Skillplug: Unsupervised skill mining for few-shot adaptation in robotic manipulation. *IEEE Robotics and Automation Letters*, 2026.

[13] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English, and Robin Rombach. Scaling rectified flow transformers for high-resolution image synthesis. In *International Conference on Machine Learning*, 2024.

[14] Futian Lab and VisIncept. Rex-M1 Preview: RoboDojo Policy Release. https://github.com/XPolicyLab/XPolicyLab/tree/main/policy/Rex_M1_preview, 2026.

[15] Yanjiang Guo, Haodong Yan, Zhide Zhong, Zhongru Zhang, Qingyuan Yang, Qingzhou Lu, Xiaoyu Chen, Yen-Jen Wang, Shuying Deng, Chenghan Yang, et al. Video prediction policy 2: Predict better, act better. *arXiv preprint arXiv:2610.10270*, 2026.

[16] Dongjie Huo, Haoyun Liu, Guoqing Liu, Dekang Qi, Zhiming Sun, Maoguo Gao, Jianxin He, Yandan Yang, Xinyuan Chang, Feng Xiong, et al. Abot-claw: A foundation for persistent, cooperative, and self-evolving robotic agents. *arXiv preprint arXiv:2604.10096*, 2026.

[17] Tao Jiang, Tianyuan Yuan, Yicheng Liu, Chenhao Lu, Jianning Cui, Xiao Liu, Shuiqi Cheng, Jiyang Gao, Huazhe Xu, and Hang Zhao. Galaxea Open-World Dataset and G0 Dual-System VLA Model. *arXiv preprint arXiv:2509.00576*, 2025.

[18] Michael Kelly, Chelsea Sidrane, Katherine Driggs-Campbell, and Mykel J. Kochenderfer. HG-DAgger: Interactive imitation learning with human experts. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 8077–8083, 2019.

[19] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024.

[20] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything. *arXiv:2304.02643*, 2023.

[21] LejuRobotics. LET: Full-size humanoid robot real-world dataset. Hugging Face dataset, 2025.

[22] Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Yang Zhaohui, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, et al. Simplevla-rl: Scaling vla training via reinforcement learning. In *International Conference on Learning Representations*, volume 2026, pages 125846–125869, 2026.

[23] Hengtao Li, Pengxiang Ding, Runze Suo, Yihao Wang, Zirui Ge, Dongyuan Zang, Kexian Yu, Mingyang Sun, Hongyin Zhang, Donglin Wang, et al. Vla-rft: Vision-language-action reinforcement fine-tuning with verified rewards in world simulators. *arXiv preprint arXiv:2510.00406*, 2025.

[24] Ruicheng Li et al. LingBot-VA: A vision-action model for robotic manipulation. *arXiv preprint*, 2026.

[25] Ruiying Li, Yunlang Zhou, YuYao Zhu, Kylin Chen, Jingyuan Wang, Sukai Wang, Kongtao Hu, Minhui Yu, Bowen Jiang, Zhan Su, et al. Roboclaw: An agentic framework for scalable long-horizon robotic tasks. *arXiv preprint arXiv:2603.11558*, 2026.

[26] Tianhong Li and Kaiming He. Back to basics: Let denoising generative models denoise. *arXiv preprint arXiv:2511.13720*, 2025.

[27] LiberAI. Liber-0 Lite: RoboDojo Policy Release. https://github.com/XPolicyLab/XPolicyLab/tree/main/policy/Liber_0_lite, 2026.

[28] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.

[29] Yicheng Liu, Zibin Dong, Baijun Ye, Tianyuan Yuan, Tao Jiang, Anqi Yang, Shicheng Cao, Haonan Liu, Yue Sun, Zihan Guo, et al. G0.5: One autoregressive stream for robot reasoning and action. *arXiv preprint arXiv:2608.11739*, 2026.

[30] Zhiyu Luo et al. Being-H0.7: A vision-language-action model with hierarchical grounding. *arXiv preprint*, 2026.

[31] Xingyu Miao, Zizun Li, Baole Fang, Kaiwen Song, Tenghui Wang, Hanxue Zhang, Yating Wang, Xudong Li, Yuping He, Xueyuan Wei, et al. InternW0-Δ: A World Action Model Bridging Predictive Dynamics and Actions with 20K+ Hours of Open Data. *arXiv preprint arXiv:2609.31394*, 2026.

[32] Yao Mu, Tianxing Chen, Shijia Peng, Zanxin Chen, Zeyu Chen, Qiangyu Feng, Zhiqian Su, Weiliang Zhou, Guodong Ma, et al. RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.

[33] NVIDIA, Johan Bjorck, et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.

[34] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. FAST: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.

[35] Physical Intelligence Team, Shanghai AI Laboratory. InternVLA-A1.5: Unifying understanding, latent foresight, and action for compositional generalization. *arXiv preprint arXiv:2607.04988*, 2026.

[36] Qwen Team. Qwen-VLA: Unifying vision-language-action modeling across tasks, environments, and robot embodiments. *arXiv preprint arXiv:2605.30280*, 2026.

[37] Qwen Team. Qwen3.5: Towards native multimodal agents, 2026. https://qwen.ai/blog?id=qwen3.5.

[38] RLinf Team. RECAP: Offline advantage-based policy optimization. RLinf Documentation, 2026. Accessed August 19, 2026.

[39] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 627–635. JMLR Workshop and Conference Proceedings, 2011.

[40] SIPAILab. Simate-beta. https://github.com/SIPAILab/simate-beta, 2026.

[41] StarVLA Community. Starvla: A lego-like codebase for vision-language-action model developing. *arXiv preprint arXiv:2604.05014*, 2026.

[42] StarVLA Community. StarVLA: A modular framework for vision-language-action research, 2026. https://github.com/StarVLA.

[43] Xiaomi Robotics Team, Jun Guo, Piaopiao Jin, Jason Li, Peiyan Li, Yingyan Li, Futeng Liu, Wanli Peng, Optimus Qin, Yifei Su, et al. Xiaomi-robotics-1: Scaling vision-language-action models with over 100k hours of real-world trajectories. *arXiv preprint arXiv:2607.15330*, 2026.

[44] Thomas Tian, Yilin Wu, and Andrea Bajcsy. Position: Good embodied reward models need bad behavior data. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 172293–172305. PMLR, 2026.

[45] Yang Tian, Yuyin Yang, Yiman Xie, Zetao Cai, et al. InternData-A1: Pioneering high-fidelity synthetic data for pre-training generalist policy. *arXiv preprint arXiv:2511.16651*, 2025.

[46] Songlin Wei, Hongyi Jing, Boqian Li, Zhenyu Zhao, Jiageng Mao, Zhenhao Ni, Sicheng He, Jie Liu, Xiawei Liu, Kaidi Kang, et al. ψ_0: An open foundation model towards universal humanoid loco-manipulation. *arXiv preprint arXiv:2603.12263*, 2026.

[47] Junjie Wu et al. LingBot-VLA: A vision-language-action model for robotic manipulation. *arXiv preprint*, 2026.

[48] Kun Wu, Chengkai Hou, Jiaming Liu, Zhengping Che, Xiaozhu Ju, Zhuqin Yang, Meng Li, Yinuo Zhao, Zhiyuan Xu, Guang Yang, et al. RoboMIND: Benchmark on multi-embodiment intelligence normative data for robot manipulation. *arXiv preprint arXiv:2412.13877*, 2024.

[49] Shihan Wu et al. RoboCOIN: An open-sourced bimanual robotic data collection for integrated manipulation. *arXiv preprint arXiv:2511.17441*, 2025.

[50] X-VLA Team. X-VLA: Soft-prompted transformer as a scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2504.20946*, 2026.

[51] Heqing Yang, Yang Yi, Liyao Wang, Linqing Zhong, Donglin Yang, Ruipu Wu, Zitong Bai, Fengjiao Chen, Manyuan Zhang, Linjiang Huang, and Si Liu. EVA-Client: A unified data collection, inference, and deployment framework for embodied policies on real robots. *arXiv preprint arXiv:2607.02646*, 2026.

[52] Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. Set-of-Mark prompting unleashes extraordinary visual grounding in GPT-4V. *arXiv preprint arXiv:2310.11441*, 2023.

[53] Tianhang Yang, Yanze Zheng, Junjie Wang, Wei-Bin Kou, Ruotong Li, and Yujiu Yang. Route by kinematics, act by observation: Kinematics-supervised expert routing in moe-augmented vla. *arXiv preprint arXiv:2607.26807*, 2026.

[54] Xingcheng Yang et al. ABot-M0: A vision-language-action model with action manifold learning for bimanual robotic manipulation. *arXiv preprint arXiv:2602.11236*, 2026.

[55] Angen Ye, Zeyu Zhang, Boyuan Wang, Xiaofeng Wang, Dapeng Zhang, and Zheng Zhu. Vla-r1: Enhancing reasoning in vision-language-action models. *arXiv preprint arXiv:2510.01623*, 2025.

[56] Cheng Yin, Wang Xu, Junpeng Yang, Sikyuen Tam, Hanyu Liu, Yuan Yao, Xiangrui Zeng, Junbo Cui, Yequan Wang, Zhouping Yin, et al. Simplememvla: A simple but effective native-video memory for vision-language-action models. *arXiv preprint arXiv:2609.05533*, 2026.

[57] Chenghao Yin et al. Genie Sim 3.0: A high-fidelity comprehensive simulation platform for humanoid robot. *arXiv preprint arXiv:2601.02078*, 2026.

[58] Checheng Yu, Chonghao Sima, Gangcheng Jiang, Hai Zhang, Haoguang Mai, Hongyang Li, Huijie Wang, Jin Chen, Kaiyang Wu, Li Chen, Lirui Zhao, Modi Shi, Ping Luo, Qingwen Bu, Shijia Peng, Tianyu Li, and Yibo Yuan. χ_0: Resource-aware robust manipulation via taming distributional inconsistencies. *arXiv preprint arXiv:2602.09021*, 2026.

[59] Ye Yuan et al. Fast-WAM: World-aware manipulation with fast action generation. *arXiv preprint*, 2026.

[60] He Zhang, Lingzhu Xiang, Haitao Lin, Zeyu Huang, Minghui Wang, Dingyan Zhong, Yubo Dong, Yihao Wu, Yongming Rao, Dongsheng Zhang, et al. Hy-embodied-0.5-vla: From vision-language-action models to a real-world robot learning stack. *arXiv preprint arXiv:2606.14409*, 2026.

[61] Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Chunyang Zhu, Jiaxing Qiu, Yuchen Yan, Jiyuan Liu, Wenhao Tang, et al. Harness vla: Steering frozen vlas into reliable manipulation primitives via memory-guided agents. *arXiv preprint arXiv:2607.08448*, 2026.

[62] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023.

[63] Yi Zhou, Connelly Barnes, Jingwan Lu, Jimei Yang, and Hao Li. On the continuity of rotation representations in neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5745–5753, 2019.